

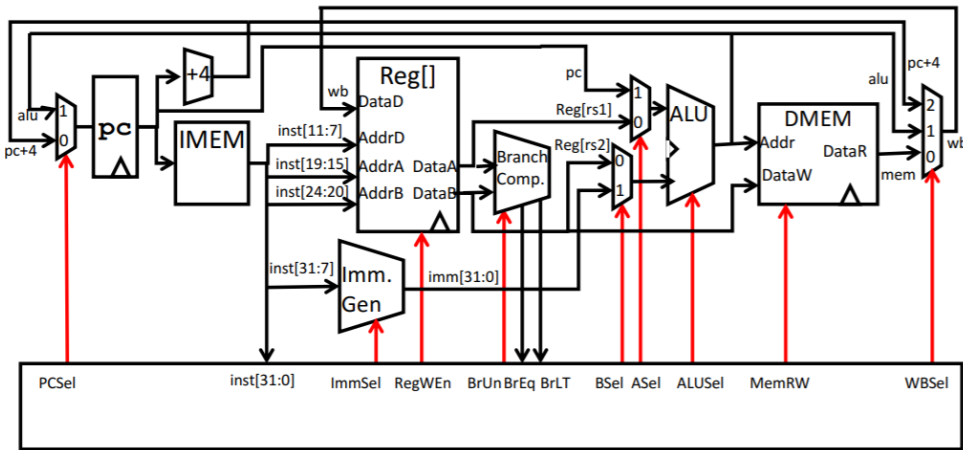
Q1 Single-Cycle Datapath

4 Points

REMINDER: Gradescope does not have submission history for in-browser assignments such as lecture questions and homeworks. If you press "save answer" or "submit" after the deadline, the autograder *will* mark it as late and you'll get 0 points in the next TCP rerun. When you review old homework and lecture assignments, do not click any buttons otherwise you risk getting a zero on that assignment.

I think we've had enough of Datapath in Lecture and Discussion (or not), but here, have some more anyways! And no, this homework is not optional.

We have reproduced the RISC-V single-cycle pipeline here.



Using the following delays for the rest of the problem. Assume any component not listed is negligible in calculations.

Element	Register clk-to-q	Register Setup	MUX	ALU	MemRead	Mem Write	RegFile Read	RegFileSetup	
Parameter	$t_{clk-to-q}$	t_{setup}	t_{mux}	t_{ALU}	$t_{MEMread}$	$t_{MEMwrite}$	t_{RFread}	$t_{RFsetup}$	1
Delay (ps)	30	20	25	200	200	250	150	20	2

Q1.1 Critical Path

1 Point

What is the critical (longest) path delay of this circuit? Your answer should be in units of ps.

The path starts at PC and ends at data write. Make sure you understand why this doesn't start from Regfile Read. Also make sure to compare the different paths taken by different instructions in determining the critical path, i.e. reading vs writing to DMEM.

Q1.2 Clock Frequency

1 Point

Now that we have determined the critical path delay, what is the maximum frequency in GHz at which this circuit can run? Please round your answer to the nearest hundredth.

Q1.3 Latency and Throughput

2 Points

Remember that latency refers to how long it takes a single instruction to finish, whereas throughput is defined as how many operations are finished per cycle. These two quantities are not the same, as we'll see when we begin to pipeline the processor.

What is the latency of a single instruction, in cycles?

What is the throughput of our datapath? (Leave your answer in instructions/cycle).

Q2

4 Points

I'm glad you think you're almost done with this problem, but you're only halfway there...

We decide to improve the performance of our CPU by splitting it into 3 stages: Instruction Fetch (IF), Execution (X), and Memory Access (M). Note that this is different from the 5-stage pipeline shown in lecture. The pipelined datapath is shown below. The blue lines indicate where we decide to make the split, meaning any necessary pipeline registers will be added to the datapath at these locations.

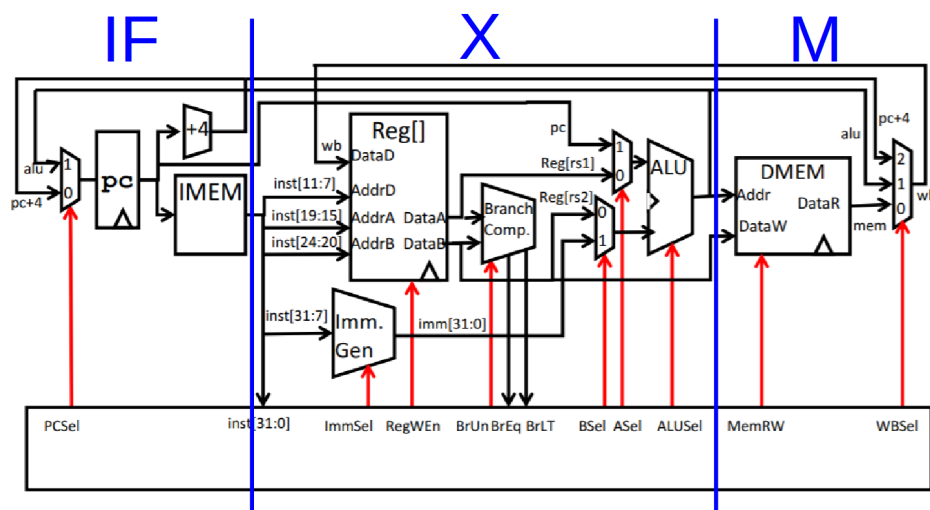
A more detailed description of each stage:

IF: the PC is input into the instruction memory and the output is saved in a pipeline register.

X: the instruction is decoded, we read data from the regfile, perform some computation with the ALU, and save any necessary data in pipeline registers.

M: data is read/written to memory, and data is potentially written back into the registers. Note that the "writing back" portion is located where the "X" operations occur, but we treat it as part of the "M" stage.

You may assume any control signal is properly handled for all pipeline stages.



The component delays are reproduced below for your convenience.

Element	Register clk-to-q	Register Setup	MUX	ALU	MemRead	Mem Write	RegFile Read	RegFileSetup	
Parameter	$t_{clk-to-q}$	t_{setup}	t_{mux}	t_{ALU}	$t_{MEMread}$	$t_{MEMwrite}$	t_{RFread}	$t_{RFsetup}$	
Delay (ps)	30	20	25	200	200	250	150	20	

Q2.1 Pipelined Critical Path Delay

1 Point

What is the critical path delay of this circuit? Your answer should be in units of ps.

475

Q2.2 Pipelined Clock Frequency

1 Point

Now that we have determined the critical path delay, what is the maximum frequency in GHz at which this circuit can run? Please write your answer as a fraction.

Q2.3 Pipelined Latency and Throughput

2 Points

Now that we have pipelined our processor, how does our throughput and latency change?

What is the latency of a single instruction, in cycles?

3

What is the throughput of our datapath? (Leave your answer in instructions/cycle).

1

Q3 Consequences of Pipelining

1 Point

Which of the following statements are true, in general, when transitioning from a single-cycle to a pipelined processor design? (Which of the followings statements are true, in general, of the pipelined design?)

☐ We can finish executing more instructions per cycle

☐ A single instruction passes through the datapath faster

☒ We can handle more than one instruction at a time

☐ We can increase the clock frequency

Q4 Pipelining Hazards

6 Points

Now, considering the execution of the following program in a standard 5-cycle CPU. Recall the 5 stages: Instruction Fetch, Instruction Decode/Register Read, ALU/Execution, Memory, and Write-back. In this case, assume that **branching is resolved in the Memory stage**.

```
1. lw s1 0(s0)
2. add s2 s1 s1
3. slli s3 s1 1
4. bne s2 s3 exit #branch not taken
5. addi t1 t0 x0
6. exit:
```

Q4.1

1 Point

Assuming read and write to registers happen in **different cycles**, **no pipelining optimizations** are applied (i.e. no forwarding, no branch prediction or pipeline flushing etc...), and branching is resolved in the **MEM stage**. How many CPU cycles will it take to execute this program on our described pipelined machine?

Q4.2

1 Point

How many stalls are there?

Q4.3

1 Point

How many data hazards are there?

Q4.4

1 Point

How many control hazards are there?

Q4.5

1 Point

Now consider this pipeline where reads and writes to register happen in the **same cycle**, that is at the start of the cycle the write occurs and

then at the end of the cycle the read occurs so we can do both a write and read in the same cycle. How many CPU cycles will this take to execute?

Q4.6

1 Point

Now consider this pipeline where reads and writes to register happen in the **same cycle** and **forwarding** is applied but there is no branch prediction. How many CPU cycles will this take to execute?

Q5 [Practice] Mover

0 Points

The (not turing complete) programming language Mover is defined as follows:

The program stores an 2-D grid of 8-bit unsigned integers, initialized to 0, a memory pointer, which starts at $(x,y) = (0,0)$, and a program flow, which starts at "FORWARD".

The program recognizes only the following 8 commands:

- $\>$ Moves the pointer one step right (+1 to x)
- $\<$ Moves the pointer one step left (-1 to x)
- $\wedge$ Moves the pointer one step up (+1 to y)
- ∇ Moves the pointer one step down (-1 to y)
- $+$ Increments the value at the pointer by 1
- $-$ Decrements the value at the pointer by 1
- $]$ If the pointer is currently pointing at a 0 and the current program flow is "FORWARD", change the program flow to "BACKWARD". Otherwise do nothing.
- $[$ If the pointer is currently pointing at a 0 and the current program flow is "BACKWARD", change the program flow to "FORWARD". Otherwise do nothing.

If the program flow is "FORWARD", then the next instruction to be executed is the one after the current one; if the program flow is "BACKWARD", then the next instruction is the one before the current instruction.

It is undefined behavior for the pointer to go outside the memory array's bounds and likewise the behavior for integer overflow and underflow are undefined. For the C version, the program halts if it reaches the end of the program string in either direction.

The language ignores any other characters in the program, and terminates if the program counter goes past the bounds of the program.

Q5.1

0 Points

You want to write a C program that interprets this language: The inputs are a valid mover program as a null terminated string, and `memory_grid`, which points to a sequence of pointers, each of which points to a buffer that is a column of the 2D grid.

```
void runMover(char* program, uint8_t** memory_grid)
{
    uint x = 0;
    uint y = 0;
    uint pc = 0;
    uint dir = 1; /* forward = true */
    uint len = strlen(program);
    while(pc >= 0 && pc < len)
    {
        switch(program[pc])
        {
            //Code for each Mover command
        }
        pc += dir ? 1 : -1;
    }
}
```

For the following operators, write the C code for the following Mover commands.

Command:

Code:

```
case '>':  
    <YOUR CODE HERE>  
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`☐ `x -= y;`☐ `pc -= x;`☐ `pc = x;`☐ `y += 1;`☐ `y += x;`☐ `pc += y;`☐ `y -= 1;`☐ `y -= x;`☐ `pc -= y;`☐ `pc = y;`☐ `x = y;`☐ `y = x;`

☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`☐ `dir |= (memory_grid[x][y] > 0);`☐ `dir |= (memory_grid[y][x] > 0);`☐ `memory_grid[x][y] += 1;`☐ `memory_grid[y][x] += 1;`☐ `memory_grid[x][y] -= 1;`☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] = pc;`☐ `memory_grid[y][x] = pc;`☐ `memory_grid[x][y] += pc;`☐ `memory_grid[y][x] += pc;`☐ `memory_grid[x][y] -= pc;`☐ `memory_grid[y][x] -= pc;`

☐ `return;`Command: `<`

Code:

```
case '<':  
    <YOUR CODE HERE>  
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`☐ `x -= y;`☐ `pc -= x;`☐ `pc = x;`☐ `y += 1;`☐ `y += x;`☐ `pc += y;`☐ `y -= 1;`☐ `y -= x;`☐ `pc -= y;`☐ `pc = y;`

☐ `x = y;`☐ `y = x;`☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`☐ `dir |= (memory_grid[x][y] > 0);`☐ `dir |= (memory_grid[y][x] > 0);`☐ `memory_grid[x][y] += 1;`☐ `memory_grid[y][x] += 1;`☐ `memory_grid[x][y] -= 1;`☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] = pc;`☐ `memory_grid[y][x] = pc;`☐ `memory_grid[x][y] += pc;`☐ `memory_grid[y][x] += pc;`

☐ `memory_grid[x][y] -= pc;`☐ `memory_grid[y][x] -= pc;`☐ `return;`Command:

Code:

```
case '^':  
    <YOUR CODE HERE>  
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`☐ `x -= y;`☐ `pc -= x;`☐ `pc = x;`☐ `y += 1;`☐ `y += x;`☐ `pc += y;`☐ `y -= 1;`☐ `y -= x;`☐ `pc -= y;`

☐ `pc = y;`☐ `x = y;`☐ `y = x;`☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`☐ `dir |= (memory_grid[x][y] > 0);`☐ `dir |= (memory_grid[y][x] > 0);`☐ `memory_grid[x][y] += 1;`☐ `memory_grid[y][x] += 1;`☐ `memory_grid[x][y] -= 1;`☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] = pc;`☐ `memory_grid[y][x] = pc;`☐ `memory_grid[x][y] += pc;`

☐ `memory_grid[y][x] += pc;`☐ `memory_grid[x][y] -= pc;`☐ `memory_grid[y][x] -= pc;`☐ `return;`

Command: ☐ `v`

Code:

```
case 'v':  
    <YOUR CODE HERE>  
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`☐ `x -= y;`☐ `pc -= x;`☐ `pc = x;`☐ `y += 1;`☐ `y += x;`☐ `pc += y;`☐ `y -= 1;`

☐ `y -= x;`☐ `pc -= y;`☐ `pc = y;`☐ `x = y;`☐ `y = x;`☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`☐ `dir |= (memory_grid[x][y] > 0);`☐ `dir |= (memory_grid[y][x] > 0);`☐ `memory_grid[x][y] += 1;`☐ `memory_grid[y][x] += 1;`☐ `memory_grid[x][y] -= 1;`☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] = pc;`

☐ `memory_grid[y][x] = pc;`☐ `memory_grid[x][y] += pc;`☐ `memory_grid[y][x] += pc;`☐ `memory_grid[x][y] -= pc;`☐ `memory_grid[y][x] -= pc;`☐ `return;`

Command:

Code:

```
case '+':  
    <YOUR CODE HERE>  
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`☐ `x -= y;`☐ `pc -= x;`☐ `pc = x;`☐ `y += 1;`☐ `y += x;`

☐ `pc += y;`☐ `y -= 1;`☐ `y -= x;`☐ `pc -= y;`☐ `pc = y;`☐ `x = y;`☐ `y = x;`☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`☐ `dir |= (memory_grid[x][y] > 0);`☐ `dir |= (memory_grid[y][x] > 0);`☐ `memory_grid[x][y] += 1;`☐ `memory_grid[y][x] += 1;`☐ `memory_grid[x][y] -= 1;`

☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] = pc;`☐ `memory_grid[y][x] = pc;`☐ `memory_grid[x][y] += pc;`☐ `memory_grid[y][x] += pc;`☐ `memory_grid[x][y] -= pc;`☐ `memory_grid[y][x] -= pc;`☐ `return;`

Command:

Code:

```
case '-':  
    <YOUR CODE HERE>  
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`☐ `x -= y;`☐ `pc -= x;`☐ `pc = x;`☐ `pc += 1;`

☐ `y += 1;`☐ `y += x;`☐ `pc += y;`☐ `y -= 1;`☐ `y -= x;`☐ `pc -= y;`☐ `pc = y;`☐ `x = y;`☐ `y = x;`☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`☐ `dir |= (memory_grid[x][y] > 0);`☐ `dir |= (memory_grid[y][x] > 0);`☐ `memory_grid[x][y] += 1;`☐ `memory_grid[y][x] += 1;`

☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] -= 1;`☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] = pc;`☐ `memory_grid[y][x] = pc;`☐ `memory_grid[x][y] += pc;`☐ `memory_grid[y][x] += pc;`☐ `memory_grid[x][y] -= pc;`☐ `memory_grid[y][x] -= pc;`☐ `return;`

Command: `]`

Code:

```
case ']':  
    <YOUR CODE HERE>  
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`☐ `x -= y;`☐ `pc -= x;`

☐ `pc = x;`☐ `y += 1;`☐ `y += x;`☐ `pc += y;`☐ `y -= 1;`☐ `y -= x;`☐ `pc -= y;`☐ `pc = y;`☐ `x = y;`☐ `y = x;`☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`☐ `dir |= (memory_grid[x][y] > 0);`☐ `dir |= (memory_grid[y][x] > 0);`

☐ `memory_grid[x][y] += 1;`☐ `memory_grid[y][x] += 1;`☐ `memory_grid[x][y] -= 1;`☐ `memory_grid[y][x] -= 1;`☐ `memory_grid[x][y] = pc;`☐ `memory_grid[y][x] = pc;`☐ `memory_grid[x][y] += pc;`☐ `memory_grid[y][x] += pc;`☐ `memory_grid[x][y] -= pc;`☐ `memory_grid[y][x] -= pc;`☐ `return;`

Command: `[`

Code:

```
case '[':
    <YOUR CODE HERE>
    break;
```

☐ `x += 1;`☐ `x += y;`☐ `pc += x;`☐ `x -= 1;`

☐ `x -= y;`☐ `pc -= x;`☐ `pc = x;`☐ `y += 1;`☐ `y += x;`☐ `pc += y;`☐ `y -= 1;`☐ `y -= x;`☐ `pc -= y;`☐ `pc = y;`☐ `x = y;`☐ `y = x;`☐ `x = pc;`☐ `y = pc;`☐ `dir &= (memory_grid[x][y] > 0);`☐ `dir &= (memory_grid[y][x] > 0);`☐ `dir |= (memory_grid[x][y] == 0);`☐ `dir |= (memory_grid[y][x] == 0);`☐ `dir &= (memory_grid[x][y] == 0);`☐ `dir &= (memory_grid[y][x] == 0);`

☐ `dir |= (memory_grid[x][y] > 0);`
☐ `dir |= (memory_grid[y][x] > 0);`
☐ `memory_grid[x][y] += 1;`
☐ `memory_grid[y][x] += 1;`
☐ `memory_grid[x][y] -= 1;`
☐ `memory_grid[y][x] -= 1;`
☐ `memory_grid[x][y] = pc;`
☐ `memory_grid[y][x] = pc;`
☐ `memory_grid[x][y] += pc;`
☐ `memory_grid[y][x] += pc;`
☐ `memory_grid[x][y] -= pc;`
☐ `memory_grid[y][x] -= pc;`
☐ `return;`

Q5.2

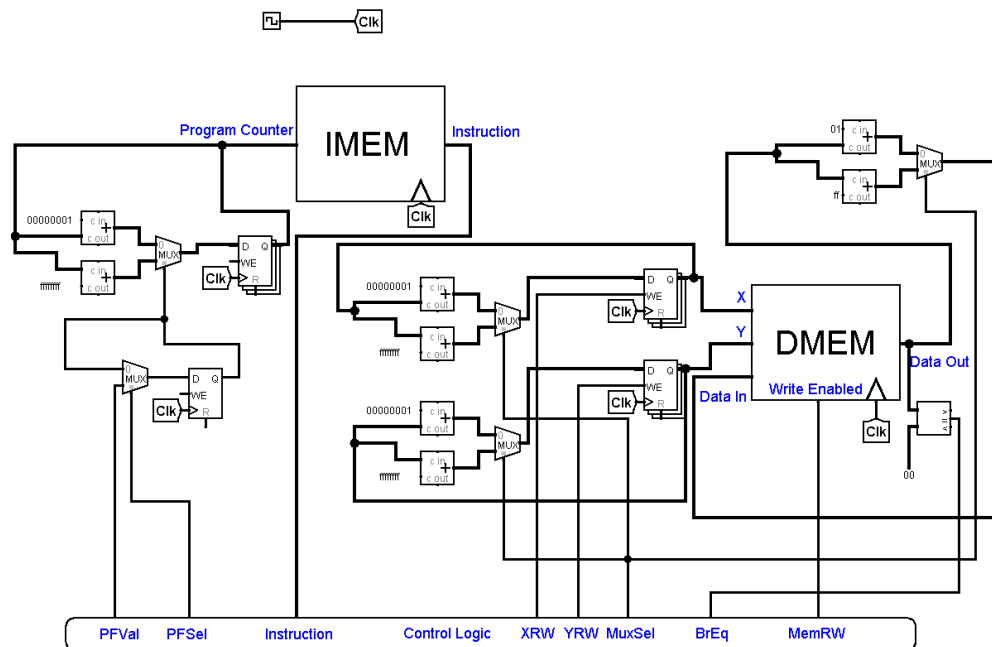
0 Points

You now want to create a circuit that runs Mover. In order to do that, you assign each Mover command a unique 4-bit code (explanations of each command have been copied for reference):

- `(0001) >` Moves the pointer one step right (+1 to `x`)
- `(1001) <` Moves the pointer one step left (-1 to `x`)
- `(0101) ^` Moves the pointer one step up (+1 to `y`)
- `(1101) v` Moves the pointer one step down (-1 to `y`)

- (0011) + Increments the value at the pointer by 1
- (1011) - Decrements the value at the pointer by 1
- (0111)] If the pointer is currently pointing at a 0 and the current program flow is "FORWARD", change the program flow to "BACKWARD". Otherwise do nothing.
- (1111) [If the pointer is currently pointing at a 0 and the current program flow is "BACKWARD", change the program flow to "FORWARD". Otherwise do nothing.

You have finished the general structure of the circuit, and just need to finish writing the control logic. The memories in question are asynchronous read but synchronous write (thus the CLK). During startup the registers are set to 0. Note that the direction register now holds zero (not one) when the program is going forward. We just run forever and it is simply undefined behavior to either overflow or underflow the PC so we aren't worrying about having to check for any of that:



Let x_3, x_2, x_1, x_0 be the bits of an instruction with x_0 being the least significant bit, and let $BrEq$ be the value of $BrEq$. Write the most simplified logical equations (fewest number of logical operators) for each output of Control Logic.

Formatting: Write your answers in **boolean form**. Do NOT add a space between $\sim$ and the variable you are negating. Add a space for everything else! (E.g. If the answer is $\text{not}((\text{not } x_0) \text{ and } x_3)$, you would

enter $\sim(\sim x_0 * x_3)$). Make sure you order variables from left to right starting with the x_0 - x_3 then any extra variable following it in alphabetical order. Do not add unnecessary parentheses or spaces between them.

PFVal

PFSel

XRW

YRW

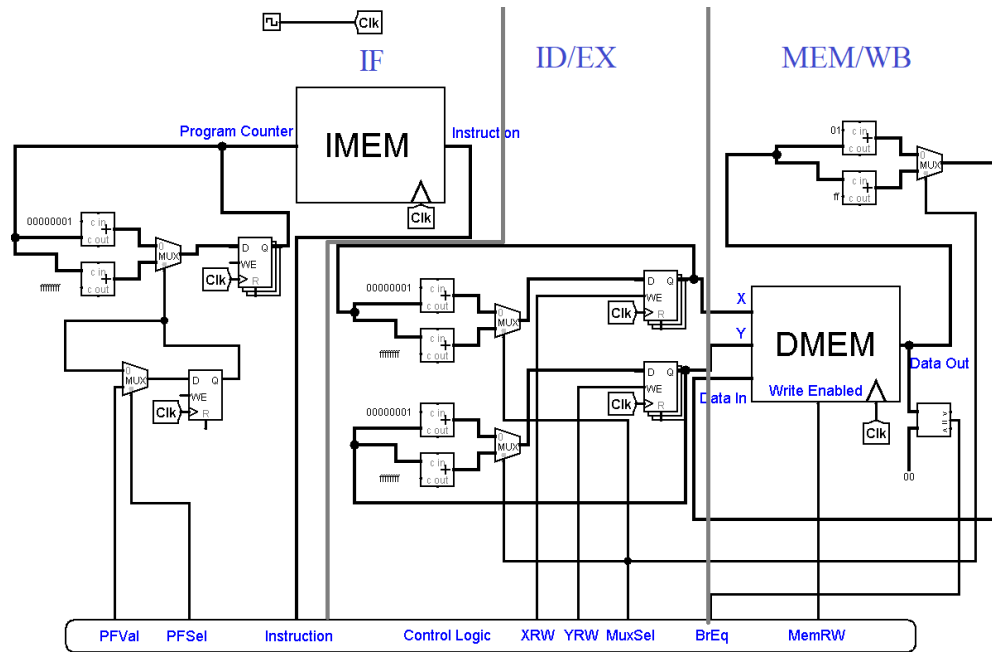
MemRW

MuxSel

Q5.3

0 Points

After finishing your circuit, you find that it's a bit slow. To speed things up, you decide to pipeline the above circuit by dividing the circuit into IF, ID/EX, and MEM/WB stages.



What hazards can occur? Assume that the single cycle datapath worked as intended (Select all that apply)

☐ Structural Hazard

☒ Control Hazard

☐ Data Hazard

☐ None of the Other Choices

Q5.4

0 Points

Please leave your answer in ns and do not add the units.

If all registers (including pipeline registers) have 2ns setup, 0ns hold, and 2ns clk->q time, the memories take 4ns to do a read and have a 2ns setup time for writes, and the sequential logic takes 0ns (yes, that's a ridiculous number, we chose it to make the path simple), what is the minimum viable clock period for the resulting datapath?

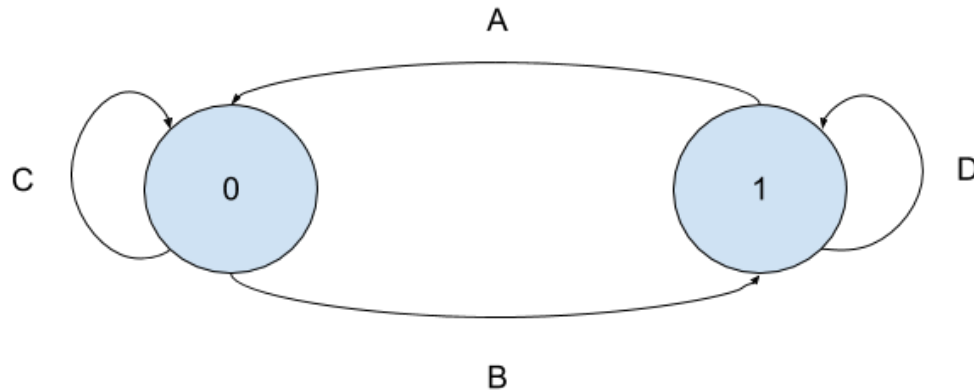
8

Q5.5

0 Points

Of course, **direction** tracking can be also implemented as a finite state machine with two states (1 = forward, 0 = backward) and 3 inputs: "CF" which is 1 when the current instruction is "[", "CB" which is 1 when the current instruction is "]", and "Mem" which is 1 if the current memory value is non-zero.

For the four transitions on the below state transition diagram, write the most simplified logical equations for each edge.



Formatting: Write your answers in **boolean form**. Do NOT add a space between $\sim$ and the variable you are negating. Add a space for everything else! (E.g. If the answer is not((not x0) and x3), you would enter $\sim(\sim x0 * x3)$). Make sure you order variables from left to right starting with the x0-x3 then any extra variable following it in alphabetical order. Do not add unnecessary parentheses or spaces between them.

A

B

C

D

HW06 - RISC-V Datapath

● UNGRADED

STUDENT

Rahul Shah

TOTAL POINTS

- / 15 pts

QUESTION 1

Single-Cycle Datapath	4 pts
1.1 Critical Path	1 pt
1.2 Clock Frequency	1 pt
1.3 Latency and Throughput	2 pts

QUESTION 2

(no title)	4 pts
2.1 Pipelined Critical Path Delay	1 pt
2.2 Pipelined Clock Frequency	1 pt
2.3 Pipelined Latency and Throughput	2 pts

QUESTION 3

Consequences of Pipelining	1 pt
----------------------------	------

QUESTION 4

Pipelining Hazards	6 pts
4.1 (no title)	1 pt
4.2 (no title)	1 pt

2/26/2021	View Submission Gradescope	
4.3	(no title)	1 pt
4.4	(no title)	1 pt
4.5	(no title)	1 pt
4.6	(no title)	1 pt
QUESTION 5		
[Practice] Mover		0 pts
5.1	(no title)	0 pts
5.2	(no title)	0 pts
5.3	(no title)	0 pts
5.4	(no title)	0 pts
5.5	(no title)	0 pts